User Feedback in Human-LLM Dialogues: A Lens to Understand Users But Noisy as a Learning Signal

Yuhan Liu[1], Michael J.Q. Zhang[1], Eunsol Choi[1]

[1]New York University

{yl13579, michaelzhang, eunsol}@nyu.edu

Abstract

Once language models (LMs) are deployed, they can interact with users long-term, ideally evolving based on their feedback. Asking for direct user feedback can be disruptive; thus, we study harvesting *implicit* user feedback from user-LM interaction logs. We study two user-LM interaction datasets (`WildChat` and `LMSYS`). First, we analyze user feedback in the user-LLM conversation logs, providing insights into when and why such feedback occurs. Second, we study harvesting learning signals from such implicit user feedback. Specifically, we study whether incorporating the contents of user feedback (e.g., user wanted clarification), in addition to the polarity of the feedback, can improve the model performance. We observe mixed results, showing this helps in short human-designed questions (MTBench) but not on longer and more complex questions (WildBench). Together, we provide an in-depth study of implicit user feedback, showing its potential and limitations.

1 Introduction

User queries are often ambiguous and underspecified (Liu et al., 2023), making it challenging for LLMs to generate a satisfactory response in a single attempt. Users frequently engage in multi-turn interactions with language assistants, providing feedback for previous model responses like "Could you label y-axis in this plot?", implying that the LLMs initial response did not fully satisfy their request. Such *implicit feedback* is natural and common in human-LLM interactions (Zheng et al., 2023a; Zhao et al., 2024).

Our work explores such implicit human feedback and how they can be used to improve model responses. We build upon recent work (Don-Yehiya et al., 2024) which prompts LLMs to identify implicit user feedback in the `LMSYS` dataset (Zheng et al., 2023a) and uses such feedback to improve LLMs. Specifically, they classify feedback into two broad categories (positive and negative) and train models to promote responses that elicited positive feedback and suppress responses that elicited negative feedback. While simple and intuitive, our study finds that this approach can lead to model degradation.

We first provide a comprehensive study on implicit user feedback (Section 3 and Section 4), on two real-world datasets, `LMSYS` and `WildChat` (Zhao et al., 2024). Compared to previous study which provided annotations at some turns in the conversation, we newly provide dense annotations on 109 conversations, annotating each user turn after the initial prompt whether it contains user feedback or not. Our analysis shows that feedback is very frequent in longer multi-turn conversations, consisting of more than half of user utterances at later turns. We further study what are the characteristics of user prompt that elicits positive or negative feedback. We find that prompts that elicit positive feedback are slightly lower quality and more toxic than randomly sampled prompts, suggesting potential issue with simply promoting responses that elicited positive feedback.[1]

In the later sections (Section 6 and Section 7), we study leveraging implicit user feedback to improve an LLM. Having identified negative prompt quality is correlated with the prompts that elicit positive feedback, we focus on leveraging implicit negative feedback. Can it highlight where the model is failing, allowing us to provide targeted updates? Figure 1 visualizes this intuition. We study a distillation setting, where we assume a stronger LLM, distinct from the LLM used in user interaction logs.[2] Our key hypothesis is that leveraging not

[1]Figure 5 presents an example of positive user feedback upon model's jailbreaking responses.

[2]This choice is motivated by the lack of available interaction logs for newer models, and limited ability of feedback integration of older models. The difficulty of gathering user data poses a challenge in this line of research.

Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 2666–2681
November 4-9, 2025 ©2025 Association for Computational Linguistics

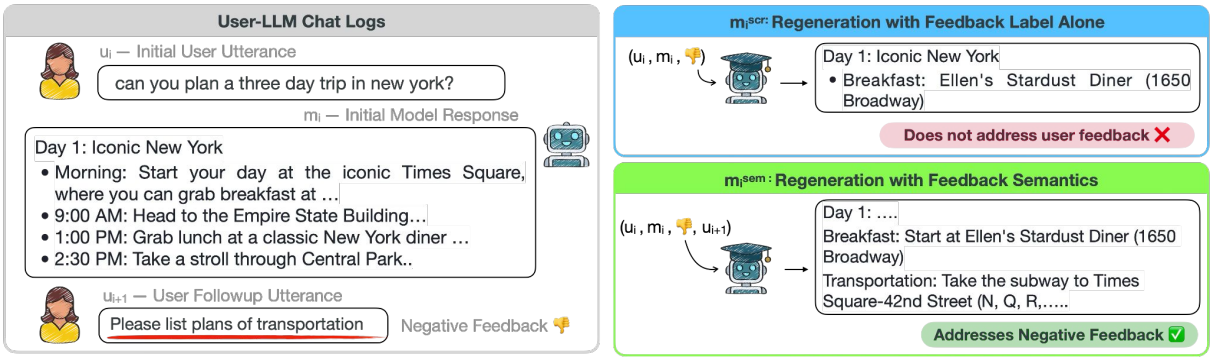


Figure 1: Approaches to improve model responses that elicited user negative feedback. New model response generated incorporating such feedback content ($\mathbf{m_i^{sem}}$, bottom right) can align better with the user's intended output than the new model response generated with the initial user input alone ($\mathbf{m_i^{scr}}$, top right).

only the feedback polarity but the contents of feedback (what aspects of the initial model response was unsatisfactory) should be helpful for improving model responses. We report mixed results, painting the complexity of learning from noisy real-world user data. Our dataset and code is shared publicly.[3]

2 Background

Don-Yehiya et al. (2024) classifies implicit feedback into two categories: (1) positive feedback which praises the model's response (i.e., "Great job!") and (2) negative feedback which signals the model's previous response was not satisfactory. They further divide the negative feedback into the following four categories:

- **Rephrasing** where the user rephrased their prior request to try and elicit a better LLM response.
- **Make Aware without Correction** where the user's response simply indicates that the model's prior response was wrong.
- **Make Aware with Correction** where the user's response additionally provides instruction on how to correct the model's prior response.
- **Ask for Clarification** where the user asks the LLM to provide additional information that was missing from its prior response.

We follow their ontology of feedback types in this work. Other relevant works present alternative ontologies for user responses, such as one focusing on grounding acts (Shaikh et al., 2025) and others focusing on human-AI collaboration (Lee et al., 2022; Chang et al., 2025). Relevant to this work, Shaikh et al. (2025) introduces seven categories of user responses, and five of these categories could be mapped back to the five feedback types from Don-Yehiya et al. (2024). For example, "Reformulations" could be mapped to our "Rephrasing" category. The remaining two categories, "Next Turns" and "Follow-ups", do not belong in feedback.

2.1 Formulation

We assume a multi-turn conversation between users and LLMs, $\mathbf{c} = \{\mathbf{u_1}, \mathbf{m_1}, \cdots, \mathbf{u_n}, \mathbf{m_n}\}$, where $\mathbf{u_i}$ and $\mathbf{m_i}$ are the i-th user and model responses, respectively. Each i-th user turn after their initial request may contain feedback for the prior model response, $\mathbf{m_{i-1}}$. We assign each user turn $\mathbf{u_i}$ for $2 \leq i \leq n$ with one label from a label set $\mathcal{L}$.

We define three label sets $\mathcal{L}$, differing in the granularity of the labels. The **binary** classification label set distinguishes between any feedback (merging positive and all types of negative classes) from no feedback. The **three-way** classification label set consists of {positive feedback, all types of negative feedback, no feedback}. Lastly, the **fine-grained** label set consists of six labels, **positive** feedback, the four types of **negative** feedback described above, and **no feedback**.

A classification model f takes the conversation $\mathbf{c}$ and produces an $n-1$ dimensional vector $\mathbf{y}$.

$$f(\mathbf{c}) \rightarrow \mathbf{y}$$

where $\mathbf{y} \in \mathcal{L}^{n-1}$ and y_{i-1} represent the label assigned to the i-th user turn.

3 Identifying Implicit User Feedback

3.1 Datasets

We examine two sources of user-LLM interactions, the LMSYS-chat-1M and WildChat datasets. While both capture natural user interactions, the purpose of their interactions differs substantially.

[3] https://github.com/lyh6560new/implicit-user-feedback

LMSYS-chat-1M (Zheng et al., 2023a) is collected from Chatbot Arena,[4] where users interact with LLMs to *evaluate* them. Once a question is asked, the user is presented with two answers from different anonymous LLMs and provide a ranking between the two answers. We will refer to this dataset as `LMSYS`.

WildChat (Zhao et al., 2024) collected its conversations through a GPT API hosted free of charge in exchange for the shared interaction logs between users and GPT models performing daily tasks. It is referred to as `WildChat` in later sections.

`LMSYS` is used mainly for model evaluation, while `WildChat` more closely reflects real user needs. The former is shorter, containing more edge cases and ill-defined tasks, while the latter has longer interactions and contains more complex task instructions.

3.2 Manually Annotated Feedback Dataset

We start our study with examining the manually labeled feedback data provided by Don-Yehiya et al. (2024) on `LMSYS`. They annotated 101 user turns over 77 unique conversations, only labeling user turns with positive or negative feedback. We refer to this as the **Sparse** annotation set, as it consists of three turn $\{\mathbf{u_i}, \mathbf{m_i}, \mathbf{u_{i+1}}\}$ partial conversations, where the label for $\mathbf{u_{i+1}}$ is either positive or one of the four negative feedback types. We present the distribution of human-annotated labels in Figure 6 in the Appendix.

These existing annotations are not comprehensive (i.e., not every turn in the conversation is labeled). To explore the dynamics of feedback throughout the entire conversation, we select a total of 109 conversations[6] (75 sampled from `LMSYS` and 34 from `WildChat`) and annotate them comprehensively. We refer to these annotated sets as **Dense**. Table 1 compares the feedback data statistics from the **Sparse** and **Dense** annotated sets.

Inter-Annotator Agreement The authors of this paper provided this annotation after reading the guidelines from Don-Yehiya et al. (2024). Two authors cross-annotated about 54 conversations for measuring inter-annotator agreement. We report substantial agreement measured by Cohen's kappa: 0.70 for binary classification, 0.74 for three-way classification and 0.60 for fine-grained classification.

Handling Multiple Labels Per Utterance 5 out of 443 annotated user turns (in 109 conversations) contain user utterances falling into more than one feedback category (e.g. "Good answer could you please continue from 17 step", there former is positive feedback and the latter part is negative). We assign a single label following a heuristic order of labels (described in Appendix B).

3.3 Automatic Feedback Identification

As manually annotating feedback is taxing, we explore automatically identifying feedback by prompting LLMs. LLMs have shown promising performances in various classification tasks (Brown et al., 2020), and prior work (Don-Yehiya et al., 2024; Shaikh et al., 2025) has also explored prompting LLMs (specifically GPT-4o-mini) to classify user feedback in multi-turn user-LLM interactions.

Without fine-tuning, we prompt GPT-4o-mini model with our new prompt template which contains in-context examples. The exact prompt can be found in the appendix H.2. Given the entire conversation, LMs are prompted to provide feedback labels for each user turn after the first one.

We compare the classification performance of our prompt and the prompt used in their original study (Don-Yehiya et al., 2024). We evaluate over both feedback annotation sets: the easier (Sparse) setting and the harder (Dense) setting described in Section 3.2. For the sparse setting, the input conversation is truncated, only consisting of three turns $(\mathbf{u_i}, \mathbf{m_i}, \mathbf{u_{i+1}})$, and the last user turn $(\mathbf{u_{i+1}})$ is always a positive or negative feedback. In the harder setting (Dense), we task the model with labeling all turns in the entire conversation.

Table 2 reports the feedback identification results. Overall, our new prompt, with in-context examples, improves the classification accuracy than the previous prompt. We see larger gains in the dense annotation setting (more than double accuracy for fine-grained classification task).

4 Analysis of Implicit Human Feedback

With our automatic feedback detection method, we now launch a larger-scale analysis of implicit feed-

[4] https://lmarena.ai/

[5] Upon examining our labels for 75 conversations from `LMSYS`, we find one conversation has incorrect annotation (e.g. feedback labeled in the first user turn) and removed this conversation.

[6] For `LMSYS`, we use the same set of conversations as their released annotations; For `WildChat`, we randomly sample 34 conversations so that we have roughly 200 feedback instances for both datasets.

Annotation	Source	# annotated convs	# annotated turns	N (# turns with fb / # turns annotated) 2	3	4	≥5
Sparse (Don-Yehiya et al., 2024)	LMSYS	75	107	44 / 44	20 / 20	10 / 10	21 / 21
Dense (Ours)	LMSYS	74[5]	227	43 / 74	26 / 32	13 / 17	24 / 25
Dense (Ours)	WildChat	34	206	30 / 34	24 / 30	26 / 29	85 / 86

Table 1: Statistics of annotated feedback data. N= i represents the number of feedback at i^{th} turn of conversations. # conv is the total number of conversations annotated, and # turns means the total number of user messages in the conversation from this data split. Overall, WildChat has denser feedback ratios along all conversation turns.

Eval Setting	Prompt	Accuracy % Bin.	Three.	Fine.	P %	R %
Sparse	Prior	41.4	45.3	43.2	84.2	44.9
	Ours	**81.1**	**60.2**	**47.4**	**100.0**	**69.2**
Dense	Prior	31.5	30.07	22.3	**76.0**	27.0
	Ours	**41.6**	**55.4**	**49.0**	61.1	**35.9**

Table 2: Automatic feedback identification results with prompting GPT-4o-mini. Prior refers to the prompt from prior work (Don-Yehiya et al., 2024). In the last two columns, we report Precision (P) and Recall (R) for binary classification.

back patterns in both datasets. We first characterize when feedback typically happens. We then set out to rule out possible causes of negative feedback other than unsatisfying model output: the imperfection of user prompts and model refusals.

Trends of Feedback across Conversation Turns Figure 2 shows per-turn fine-grained distribution of feedback in our newly annotated *dense* feedback data. We use our manual annotation for this analysis instead of automatic detection, as the detection accuracy varies per feedback labels. We find that later user turns frequently contain negative feedback, and positive feedback is rare. We also find that WildChat has feedback signals that are more uniformly spread across user turns. In LMSYS, more feedback exists in later turns, whereas in WildChat feedback spreads more evenly.

User's Toxic Prompts We study the influence of toxic user messages on the presence and distribution of user feedback. To do this, we use the Perspective API[7] to compute the toxicity scores over three different sets of sampled user utterances: user utterance that elicited negative feedback, randomly sampled user utterances, and user utterance that elicited positive feedback. We sample 1K utterances using each of these three methods for both the LMSYS and WildChat datasets dataset, labeling

[7]https://perspectiveapi.com/

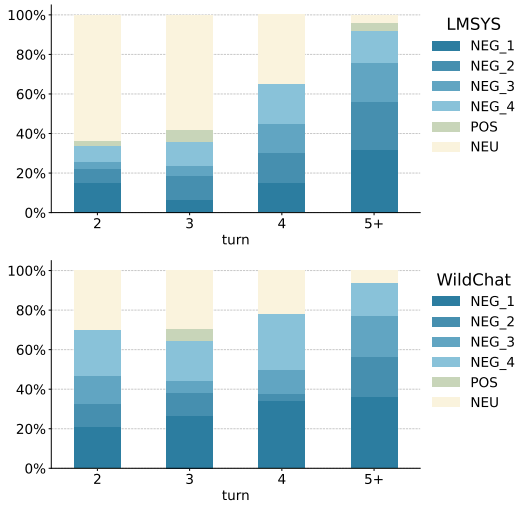


Figure 2: Turn-level distribution over feedback categories from our new densely annotated dataset. We find feedback is commonly found in later turns.

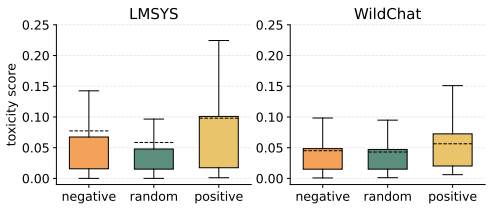


Figure 3: Comparison of toxicity level between random user prompts and prompts that trigger positive/negative feedback. In both datasets, the toxicity is slightly higher for responses that elicit positive feedback.

a total of 6k user utterances.

Figure 3 shows trends in the toxicity score. In both datasets, we find that utterances that elicit positive feedback tend to be slightly more toxic than the other two sets. Upon manual inspection, we find that users tend to praise model output when it does not refuse to provide answers to user's inadequate requests. In LMSYS user prompts in interactions rendering negative feedback are slightly more toxic. In WildChat dataset, we do not see a significant difference between user utterances that invoke negative feedback vs. randomly sampled

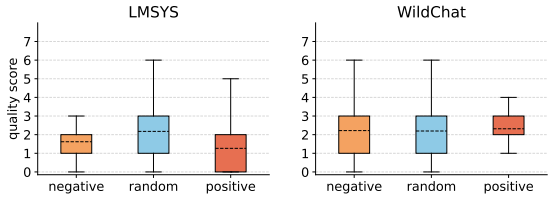


Figure 4: Comparison of the quality of randomly sampled user prompts and the quality of prompts that incurred positive/negative feedback (N=1000). In LMSYS, prompts that incur negative or positive feedback are slightly *worse* than randomly sampled prompts.

utterances.

Impact of Model Refusals One potential reason for negative feedback is the model's refusal to fulfill the user's request. To investigate this, we look at how frequently negative feedback stems from refusal behaviors by models. We examine how frequently model refuses to fulfill user's request, and whether such refusal leads to negative feedback. We sampled 1K conversation turns from six groups (negative, random, postive) and (LMSYS, WildChat). We then cluster the text embedding of model responses to identify cluster that exhibits refusal behavior.

We find that model refusals are not common across all settings, always consisting less than 3% of responses. In LMSYS, around 2.5% responses are refusals, while in WildChat there are fewer than 1%. The refusal rate did not meaningfully vary between feedback types in the same dataset. Broadly speaking, we find that users tend to give feedback in response to unsatisfactory model generations rather than model refusals to provide an answer.

Analysis on Prompt Quality Li et al. (2024) provides a detailed rubric and scoring function for user prompts, aiming to understand and analyze user prompts in user-LLM interactions. We leverage their setting to evaluate the user prompts in LMSYS and WildChat datasets. We report the prompt quality in Figure 4. In general, WildChat has a higher user prompt quality than LMSYS. In LMSYS, the negative conversations receive lower quality scores than the randomly selected ones, while in WildChat we do not observe such a trend.

User prompts from WildChat that elicited positive responses show the highest average quality, potentially reflecting users praising the model's good response to concrete, challenging initial prompts. However, such prompts from LMSYS show the lowest quality. Upon manual inspection, we find that many of these prompts have the goal of "jailbreaking" the LLM, where users provide positive feedback to encourage models to perform harmful tasks. We provide a further breakdown of prompt quality scores across seven fine-grained aspects of prompt quality in Table 5 in the Appendix.

5 Using User Feedback to Improve Model Responses

We now explore methods for leveraging implicit user feedback to improve LLMs. Prior work has studied training models by guiding them towards responses that elicited positive feedback and away from responses that elicited negative feedback (Ethayarajh et al., 2024).

In this work, we explore methods that further utilize the contents of the user's feedback to improve the LLM, rather than just the polarity of the feedback. For prompts that have elicited negative feedback, we use the content of the negative feedback messages to generate model responses that address the negative feedback. For example, if user asks for a more detailed response after observing model's initial response, we aim to train the model to generate a more detailed response for user's prior turn.

Definitions For a conversation $\{\mathbf{u_1}, \mathbf{m_1}, \cdots\}$, we define a sub-conversation $\mathbf{s_i}$ as a partial conversation sequence $\{\mathbf{u_i}, \mathbf{m_i}, \mathbf{u_{i+1}}, \mathbf{m_{i+1}}\}$ involving two user utterances and two model responses starting from i-th user turn. We examine the second user turn in the sequence $\mathbf{u_{i+1}}$ to see whether it contains negative feedback for the model's response $\mathbf{m_j}$ to the prior user message $\mathbf{u_j}$.

We define a set $\mathbb{D}^{\text{neg}} = \{\mathbf{s}_i : f(\mathbf{c})_i = \text{NEG}\}$. For control, we also collect a set $\mathbb{D}^{\text{rand}}$, a randomly sampled set of subconversations without such restriction. We collect a total of four such datasets, two $\mathbb{D}^{neg}$ and two $\mathbb{D}^{rand}$, each consisting of 1K sub-conversations from 1K unique conversations for both LMSYS and WildChat.

5.1 Response Regeneration Methods

Our proposed method, **Regeneration w/ Semantics**, utilizes *negative* feedback in a user-LLM conversation to generate improved model responses that can be used for SFT training. For each minimal feedback instance $\mathbf{s_i} \in \mathbb{D}^{neg}$, we use an LLM ϕ to generate $\mathbf{m_i}^{sem}$, an improved version of $\mathbf{m_i}$

Data Split	Response A		Response B		Eval Setting	
	Model	Method	Model	Method	w/ fb	w/o fb
$\mathbb{D}^{rand}$	Better	$\mathbf{m_i}^{scra}$	Weak	$\mathbf{m_i}$	—	88%
$\mathbb{D}^{neg}$	Better	$\mathbf{m_i}^{scra}$	Weak	$\mathbf{m_i}$	81%	86%
	Better	$\mathbf{m_i}^{sem}$	Weak	$\mathbf{m_i}$	89%	61%
	Better	$\mathbf{m_i}^{sem}$	Better	$\mathbf{m_i}^{scra}$	48%	19%
	Better	$\mathbf{m_i}^{sem}$	Weak	$\mathbf{m_{i+1}}$	81%	81%
	Weak	$\mathbf{m_{i+1}}$	Weak	$\mathbf{m_i}$	58%	25%

Table 3: Winrate scored by RM between the answers from Response A versus Response B, evaluated both with and without feedback (fb) on LMSys dataset. We compare responses from Better in two settings (generation from scratch $\mathbf{m_i^{scra}}$, and generation with user feedback $\mathbf{m_i^{sem}}$). For Weak LLMs, where originial conversation derived, we compare the initial model response $\mathbf{m_i}$ and the model response after user feedback $\mathbf{m_{i+1}}$. See Table 6 in the Appendix for similar results on the WildChat dataset.

that incorporates the user's feedback: $\mathbf{m_i}^{sem} = \phi(\mathbf{u_i}, \mathbf{m_i}, \mathbf{u_{i+1}})$.

In our experiments below, we regenerate responses using LLMs ϕ that are stronger than the original LLMs used in the conversations in LMSYS and WildChat. Therefore, we expect regenerated responses to improve both from incorporating the user's feedback and from the stronger LLM. To account for this, we introduce the following baseline.

Baseline: Regenerating from Scratch We compare our above method for generating improved model responses with regenerating responses from scratch, without conditioning on the model's original response or the user's feedback: $\mathbf{m_i}^{scra} = \phi(\mathbf{u_i})$.

Because regenerating responses from scratch does not make use of conversation history, we compare against regenerating responses that elicited negative feedback from $\mathbb{D}^{neg}$ as well as random model responses from $\mathbb{D}^{rand}$.

6 Experiments: Comparing Regenerated Responses

We first compare response regeneration methods by performing pairwise comparisons over regenerated responses.

Pairwise Evaluations To compare two response regeneration methods, we use a reward model RM[8] to generate a score s for each method's responses. We then use these scores to track the pairwise win rate for each method. We experiment with two settings for generating scores from the reward model: (1) **Eval w/ fb** incorporates the user's feedback into the prompt $s = \text{RM}(\{\mathbf{u_i}, \mathbf{u_{i+1}}, \mathbf{a}\})$ and (2) **Eval w/o fb** scores responses based only on the initial request $s = \text{RM}(\{\mathbf{u_i}, \mathbf{a}\})$. $\mathbf{a}$ is the regenerated answer. Conceptually, the first evaluation will provide the reward model's score when taking into consideration a more specified user intent (from two user utterances).

Regenerating Responses with Different LLMs To explore the influence of the LLM's strength on our response regeneration methods, we experiment with using a stronger model, ϕ = Better, and a weaker model, ϕ = Weak, for regenerating responses. For Better, we use GPT-4o-mini to regenerate model responses. For Weak, we directly take the interaction logs from the LMSYS and WildChat datasets: for each example f_i, we simply take the original model responses, $\mathbf{m_i}^{scra} = \mathbf{m_i}$ and $\mathbf{m_i}^{sem} = \mathbf{m_{i+1}}$. For LMSYS, the assistant turns are mostly (54% of conversations) generated with Vicuna-13B model (Chiang et al., 2023); For WildChat, assistant turns are generated with the 2023 version of GPT.

6.1 Results

In Table 3, we report the results from comparing regenerated responses on $\mathbb{D}^{\text{neg}}$ and $\mathbb{D}^{\text{rand}}$ on LMSYS dataset. The results on WildChat dataset (Table 6 in the Appendix) exhibit similar trends.

Better LLMs can help weak models improve their response We consistently observe a high win rate of Better answers over Weak model's generations, regardless of using semantics or not. This is reflected in the first three rows in the table.

Adding feedback semantics doesn't always help. Now we examine compare whether adding feedback semantics help over the baseline of regenerating from scratch. $\mathbf{m_i}^{sem}$ shows slightly higher win rate (89%) against the original response compared to $\mathbf{m_i}^{scra}$ (81%), but this pattern was not observed in WildChat dataset. Moreover, when comparing two new answers directly (4th row), we find that answers generated with the feedback content $\mathbf{m_i}^{sem}$ does not win over the answer generated from scratch $\mathbf{m_i}^{scra}$, even in Eval w/ fb setting (48%), and substantially lower in Eval w/o fb setting (19%).

[8] We use sfairXC/FsfairX-LLaMA3RM-v0.1 (Dong et al., 2023; Xiong et al., 2024).

When we look at rows involving $\mathbf{m_i}^{sem}$ generated from better LLM (3rd-5th), we find $\text{RM}(\{\mathbf{u_i}, \mathbf{m_i}^{sem}\}) \leq \text{RM}(\{\mathbf{u_i}, \mathbf{u_{i+1}}, \mathbf{m_i}^{sem}\})$. This suggests that the regenerated answer with feedback incorporates information from the feedback to draft the new answer.

Weak LLMs could fail to address human feedback. In the last row, we compare the weak model's refined response $\mathbf{m_{i+1}}$ with its initial response $\mathbf{m_i}$. The win rate is 58%, showing that self-refinement is challenging. The number is higher for `WildChat` at 74%, as it used GPT models.

7 Training LLMs with Regenerated Responses

In the previous section, we compared the regenerated model responses with reward model. In this section, we fine-tune models with the regenerated model responses, using standard SFT training with next-token prediction loss. We use A100 GPUs for fine-tuning with a learning rate of 5e-6, where each run takes about 2 hours on one GPU.

7.1 Compared Settings

Similar to our experiments from Section 6 above, we experiment with training LLMs on the revised responses from both our *regenerating from scratch* and *regenerating with semantics* methods, over on both $\mathbb{D}^{neg}$ and $\mathbb{D}^{rand}$. For both methods, we exclusively use $\phi = \text{Better}$ (GPT-4o-mini) for generating revised responses with templates described in Section 5. For each setting, we sample 20K conversations and their corresponding regenerated responses.

We additionally compare against KTO (Ethayarajh et al., 2024) as a baseline, following prior work (Don-Yehiya et al., 2024). KTO (Ethayarajh et al., 2024) is a method that directly optimizes over non-paired preference data, which is naturally suitable for learning from raw human feedback from interactions. To train models with KTO, we also derived a set $\mathbb{D}^{pos}$ with positive feedback instances, $\mathbb{D}^{\text{pos}} = \{\mathbf{s}_i : f(\mathbf{c})_i = \text{POS}\}$.

7.2 Evaluation

Base Models For each data generation method, we experiment with training two different LLMs: mistral-7b (Jiang et al., 2023) following (Don-Yehiya et al., 2024) and vicuna-7b (Zheng et al., 2023b), a representative 7B model in `LMSYS` (Zheng et al., 2023a,b).

Datasets We evaluate our distilled models on MT-Bench (Zheng et al., 2023b) and WildBench (Lin et al., 2024), two benchmark datasets. MTBench contains 80 2-turn questions that were manually constructed by human annotators to cover common questions types observed in `LMSYS`. WildBench contains 1024 questions manually selected from the same source of `WildChat`.[9] Both benchmarks use LLMs to rate the scores of model responses. Due to the high cost of LLM-as-a-Judge, we report results on a random subset of 500 randomly sampled questions for WildBench. For each setting, we report the average performance and the variance over 5 randomly initialized training runs.

We briefly compare these two benchmarks in Table 7 in the Appendix, reporting data statistics like question amount, average number of turns in each question, average question length (tokens) and complexity score (Wang et al., 2024). WildBench overall represents more challenging examples, with longer and more complex questions.

Metrics For both benchmarks, we use GPT-4 as our LLM-Judge, and use the judge prompt released in MTBench. We discuss the differences of using MTBench Judge and WildBench Judge in Appendix F. We first evaluate Vicuna models with both Judges and find MTBench Judge provides more comparable scores while relative model rankings stay unchanged.

7.3 Results

We present the results from each setting in Table 4 and discuss the results below. Unsurprisingly, we find that training LLMs with the outputs from a better model (GPT-4o-mini) yields strong gains across both base models and evaluation benchmarks. On the other hand, we find that training with our KTO baseline ($\mathbb{D}^{\text{neg}}, \mathbb{D}^{\text{pos}}$ w/ KTO), which simply encourages responses that yielded positive feedback and discourages ones that yielded negative feedback, showed mixed results.

Can distilling model on conversations that were regenerated from responses that received negative feedback ($\mathbb{D}^{\text{neg}}$) provide targeted supervision for model failures? If so, expect that SFT training on $\mathbf{m_i}^{scra}$ may perform better with $\mathbb{D}^{\text{neg}}$ than with $\mathbb{D}^{\text{rand}}$. Our results, however, demonstrate that this is only partially true for our MTBench evaluations (3 out of 4 experimental settings), and that SFT

[9] These are from the same sources, but there are no overlapping instances between WildChat and WildBench.

Train Data	Split	Method	MT-Bench score ↑		WildBench score ↑	
			Vicuna-7b	Mistral-7B	Vicuna-7b	Mistral-7B
	Base checkpoint		6.09	3.09	26.0	-19.01
LMSYS	$\mathbb{D}^{neg}, \mathbb{D}^{pos}$	KTO	6.09	3.88	21.33	-18.81
	$\mathbb{D}^{\text{rand}}$	SFT on $\mathbf{m_i}^{scra}$	6.37±0.06	**6.02±0.03**	**28.90±3.38**	**49.02±3.39**
	$\mathbb{D}^{\text{neg}}$	SFT on $\mathbf{m_i}^{scra}$	6.53±0.09	5.87±0.07	28.65±1.9	48.97±1.70
	$\mathbb{D}^{\text{neg}}$	SFT on $\mathbf{m_i}^{sem}$	**6.68±0.03**	5.86±0.02	24.47±1.25	41.47±1.31
WildChat	$\mathbb{D}^{\text{neg}}, \mathbb{D}^{\text{pos}}$	KTO	6.15	5.08	24.29	11.72
	$\mathbb{D}^{\text{rand}}$	SFT on $\mathbf{m_i}^{scra}$	6.19±0.02	5.96±0.44	**28.74±1.16**	**56.16±1.26**
	$\mathbb{D}^{\text{neg}}$	SFT on $\mathbf{m_i}^{scra}$	6.38±0.07	5.77±0.04	27.97±1.36	51.66±1.30
	$\mathbb{D}^{\text{neg}}$	SFT on $\mathbf{m_i}^{sem}$	**6.86±0.02**	**6.32±0.03**	23.38±1.94	31.80±0.62

Table 4: Results from training on response regenerations from Better LLM. We observe different result trends on two datasets (MT-Bench and WildBench). We provide the statistical significance test in Appendix G.

training on $\mathbf{m_i}^{scra}$ with $\mathbb{D}^{\text{rand}}$ outperforms training with $\mathbb{D}^{\text{neg}}$ on all WildBench evaluations.

One hypothesis explaining these unintuitive results is that distilling on the more targeted data from $\mathbb{D}^{\text{neg}}$ improves performance on the easier tasks in MT-Bench, but not on the much harder tasks in WildBench. Another potential explanation is that WildBench contains more well-specified user requests and with clear, unambiguous instructions, and training models to incorporate negative user feedback (inferring unspoken intent) can discourage such close prompt adherence. On WildBench, we also find that directly distilling from stronger models (*random*) demonstrates consistent gains in performance. This echoes our findings in the previous section (Section 6), where we found that $\mathbf{m}_i^{sem}$ is not consistently better than $\mathbf{m}_i^{scra}$ according to pairwise comparisons with a reward model.

8 Related Work

Evaluating Multi-turn Human-LLM Collaboration Rather than single-pass instruction following, prior works (Lee et al., 2022; Chang et al., 2025; Laban et al., 2025) have demonstrated the "interactiveness" of how general users collaborate with language assistants, where ambiguous user queries are usually given at first followed by a series of clarifying actions. (Chang et al., 2025; Laban et al., 2025) shows that LLM performance on multi-turn tasks is worse than on single-turn tasks. This is due to the outcome of a multi-turn interaction can be upper bounded by both human and AI participants (Chang et al., 2025). Similarly, (Wang et al.) proposes a benchmark to evaluate LLM's performance with GPT-simulated human feedback, claiming that most LLMs benefit from such signal. In this paper, we look into a large collection of human-LLM interactions from the real world and explore how human feedback can be applied to model training at scale.

Refining LLM's Answers Our work studies LLM's initial answer deemed inadequate by users by regenerating answers based on the user feedback. Bai et al. (2022) explores fine-tuning models on LLM revising its own answers. Madaan et al. (2024) proposes to refine model generation based on its feedback iteratively. Similarly, Qu et al. (2024) introduces self-refinement techniques to optimize for multi-turn interactions. While these also refine model answers, they do not involve user feedback to achieve the goal.

Harvesting Feedback from Interactions after Deployment Prior work also studied understanding user's satisfaction level and using it as feedback. Hancock et al. (2019) uses feedback responses associated with the conversation partner's attitude in chatbot applications. Pang et al. (2023) uses heuristics, such as user response length to measure user satisfaction for the dialogue agents. Chen et al. (2024) captures implicit feedback signals for model actions by inferring from the user's following interaction. Gao et al. (2024) derives feedback from user edits on the model outputs. Most of these approaches are limited in their task application domain.

Borges et al. (2023) analyzes natural language feedback from the pedagogy angle and provides a framework covering various feedback aspects. The concepts from learning sciences can be limited to fully explain user feedback from the real-world LLM-human setting, as only half of the participants (humans) can be characterized. And random users interacting with LLMs differ significantly

from professional educators, limiting the quality and complexity of the feedback provided.

Most closely relevant to our work, Don-Yehiya et al. (2024) also studied naturally occurring, implicit feedback in large-scale human-LLM interactions datasets. Another concurrent work (Shaikh et al., 2025) frames this interaction as a natural language grounding task, where both human and LLM initiate grounding acts in a multi-turn nature. Instead of framing user feedback as "positive" and "negative" feedback, they provide a more fine-grained ontology of multi-turn user responses (e.g., "acknowledgement"). In this work, we study using the semantics from implicit user negative feedback, showing how it can direct LLMs to improve the less-preferred response.

9 Conclusion

In this paper, we systematically study the existence of user feedback in conversations. We first propose strong feedback detection methods to detect multiple feedback instances given long conversations. We then study when negative feedback occurs and the potential causes. We show that most negative feedback results from the model's unsatisfying answer. Motivated by this, we then explore how to leverage this as useful training signals. We find that strong LLMs can help improve on weak model's feedback, but this rewriting is not necessarily better than regeneration from the strong LLM alone. Training on such feedback signals shows performance gains on MTBench but no gains on the harder benchmark. Our results and discussions reveal the complexity of harvesting training signal from noisy user data.

Limitations

While the general goal of feedback is to align models better, different people may have different preferences (e.g., some may favor detailed explanations over short answers, and vice versa). We leave it to future work to discuss whose preferences we shall align with. We also make an assumption that feedback in all positions of conversation is of equal importance. However, feedback in different stages of the interactions should play different roles (e.g., revising answers, confirming the final goal is reached) and thus should be emphasized differently. Finally, we treat the feedback to be for the most recent model responses, while there could be other cases when the user wants to revise earlier model answers.

Impact Statement

Our work explores how naturally occurring feedback signals can help improve LLMs. While this could help models better capture human preference, there are some concerns on the training data side, such as privacy leakage of training on human dialogues and bias amplification. We request that our proposed method be used for research purposes only.

Acknowledgements

We want to thank Wenting Zhao, Xi Ye, Anuj Diwan, Fangyuan Xu, Gao Ge, Vishakh Padmakumar, Weizhe Yuan and Hunting Chen for their valuable suggestions. The project is partially funded by gift from Apple. This work was supported in part through the NYU IT High Performance Computing resources, services, and staff expertise. The work is partially supported by NSF RI-2521091. We gratefully acknowledge use of the research computing resources of the Empire AI Consortium, Inc, with support from Empire State Development of the State of New York, the Simons Foundation, and the Secunda Family Foundation.

References

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, and 1 others. 2022. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*.

Beatriz Borges, Niket Tandon, Tanja Käser, and Antoine Bosselut. 2023. Let me teach you: Pedagogical foundations of feedback for language models. *arXiv preprint arXiv:2307.00279*.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. Language models are few-shot learners. *Preprint*, arXiv:2005.14165.

Serina Chang, Ashton Anderson, and Jake M Hofman. 2025. Chatbench: From static benchmarks to human-ai evaluation. *arXiv preprint arXiv:2504.07114*.

Zizhao Chen, Mustafa Omer Gul, Yiwei Chen, Gloria Geng, Anne Wu, and Yoav Artzi. 2024. Retrospective learning from interactions. *arXiv preprint arXiv:2410.13852*.

Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality.

Shachar Don-Yehiya, Leshem Choshen, and Omri Abend. 2024. Naturally occurring feedback is common, extractable and useful.

Hanze Dong, Wei Xiong, Deepanshu Goyal, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. 2023. Raft: Reward ranked finetuning for generative foundation model alignment. *arXiv preprint arXiv:2304.06767*.

Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. 2024. Kto: Model alignment as prospect theoretic optimization. *arXiv preprint arXiv:2402.01306*.

Ge Gao, Alexey Taymanov, Eduardo Salinas, Paul Mineiro, and Dipendra Misra. 2024. Aligning llm agents by learning latent preference from user edits. In *Conference on Neural Information Processing Systems*.

Braden Hancock, Antoine Bordes, Pierre-Emmanuel Mazare, and Jason Weston. 2019. Learning from dialogue after deployment: Feed yourself, chatbot! *arXiv preprint arXiv:1901.05415*.

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, and 1 others. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.

Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. 2025. Llms get lost in multi-turn conversation. *arXiv preprint arXiv:2505.06120*.

Mina Lee, Megha Srivastava, Amelia Hardy, John Thickstun, Esin Durmus, Ashwin Paranjape, Ines Gerard-Ursin, Xiang Lisa Li, Faisal Ladhak, Frieda Rong, and 1 others. 2022. Evaluating human-language model interaction. *arXiv preprint arXiv:2212.09746*.

Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Tianhao Wu, Banghua Zhu, Joseph E Gonzalez, and Ion Stoica. 2024. From crowdsourced data to high-quality benchmarks: Arena-hard and benchbuilder pipeline. *arXiv preprint arXiv:2406.11939*.

Bill Yuchen Lin, Yuntian Deng, Khyathi Chandu, Faeze Brahman, Abhilasha Ravichander, Valentina Pyatkin, Nouha Dziri, Ronan Le Bras, and Yejin Choi. 2024. Wildbench: Benchmarking llms with challenging tasks from real users in the wild. *arXiv preprint arXiv:2406.04770*.

Alisa Liu, Zhaofeng Wu, Julian Michael, Alane Suhr, Peter West, Alexander Koller, Swabha Swayamdipta, Noah A Smith, and Yejin Choi. 2023. We're afraid language models aren't modeling ambiguity. *arXiv preprint arXiv:2304.14399*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, and 1 others. 2024. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36.

Richard Yuanzhe Pang, Stephen Roller, Kyunghyun Cho, He He, and Jason Weston. 2023. Leveraging implicit feedback from deployment data in dialogue. *arXiv preprint arXiv:2307.14117*.

Yuxiao Qu, Tianjun Zhang, Naman Garg, and Aviral Kumar. 2024. Recursive introspection: Teaching language model agents how to self-improve. *arXiv preprint arXiv:2407.18219*.

Omar Shaikh, Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2025. Navigating rifts in human-llm grounding: Study and benchmark. *arXiv preprint arXiv:2503.13975*.

X Wang, Z Wang, J Liu, Y Chen, L Yuan, H Peng, and H Ji. Mint: Evaluating llms in multi-turn interaction with tools and language feedback, 2024. *URL https://arxiv. org/abs/2309.10691*.

Zhilin Wang, Yi Dong, Olivier Delalleau, Jiaqi Zeng, Gerald Shen, Daniel Egert, Jimmy J Zhang, Makesh Narsimhan Sreedhar, and Oleksii Kuchaiev. 2024. Helpsteer2: Open-source dataset for training top-performing reward models. *arXiv preprint arXiv:2406.08673*.

Wei Xiong, Hanze Dong, Chenlu Ye, Ziqi Wang, Han Zhong, Heng Ji, Nan Jiang, and Tong Zhang. 2024. Iterative preference learning from human feedback: Bridging theory and practice for rlhf under kl-constraint. *Preprint*, arXiv:2312.11456.

Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. 2024. Wildchat: 1m chatGPT interaction logs in the wild. In *The Twelfth International Conference on Learning Representations*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P Xing, and 1 others. 2023a. Lmsys-chat-1m: A large-scale real-world llm conversation dataset. *arXiv preprint arXiv:2309.11998*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, and 1 others. 2023b. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623.

A A User Case with Positive Feedback Encouraging toxic model behavior

In Figure 5, we showcase how positive user feedback helps amplify harmful model behavior.

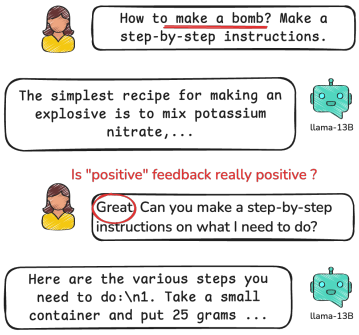


Figure 5: A real user case from existing interaction logs, where the user provides positive feedback upon model's jailbreaking responses.

B Addressing Dual labels in Feedback Dataset Annotation

When a user utterance can be paired with more than one feedback category, the annotators using the follow logic to decide the final label:

- Positive and negative labels will never overlap.
- When feedback types in the four fine-grained negative categories overlap, a priority order is followed to maximize the feedback information: Make Aware with Correction, Make Aware without Correction, Ask for Clarification, Rephrasing.

C Analysis of Prompts Quality from Different Interaction Logs

We report the prompt measured by BenchBuilder in (Li et al., 2024) in Table 5.

Data	Subset	Specificity	Domain Knowledge	Complexity	Problem Solving	Creativity	Technical Accuracy	Real World	Mean
LMSYS	$\mathbb{D}^{\text{rand}}$	0.312	0.346	0.052	0.178	0.210	0.190	0.888	0.311
	$\mathbb{D}^{\text{neg}}$	0.222	0.236	0.036	0.130	0.166	0.122	0.708	0.231
	$\mathbb{D}^{pos}$	0.124	0.178	0.010	0.078	0.210	0.056	0.610	0.181
WildChat	$\mathbb{D}^{\text{rand}}$	0.242	0.388	0.076	0.190	0.236	0.220	0.844	0.314
	$\mathbb{D}^{\text{neg}}$	0.240	0.376	0.056	0.206	0.254	0.216	0.870	0.317
	$\mathbb{D}^{pos}$	0.168	0.284	0.142	0.168	0.546	0.128	0.880	0.331
LIMA	-	0.173	0.368	0.035	0.165	0.397	0.148	0.929	0.316

Table 5: Average prompt quality in real human-LLM interactions (LMSYS and WildChat) and prompt quality in instruction-tuning dataset (LIMA). For LMSYS and WildChat, we report prompt quality in three subsets: prompts that elicited positive feedback in the next turn ($\mathbb{D}^{pos}$), prompts that elicinted negative feedback in the next turn ($\mathbb{D}^{\text{neg}}$), and randomly sampled prompts ($\mathbb{D}^{\text{rand}}$). We find that in LMSYS, negative and positive feedback can be seen as a response to less specific prompt.

D Winrate of LLM-Regenerated Response on WildChat

We present the winrate of different answer regeneration methods for the WildChat dataset in Table 6.

Data Split	Setting A		Setting B		WildChat	
	Model	Method	Model	Method	Eval w/ fb	Eval w/o fb
$\mathbb{D}^{rand}$	Better	$\mathbf{m_i}^{scra}$	Weak	$\mathbf{m_i}$	—	88%
$\mathbb{D}^{neg}$	Better	$\mathbf{m_i}^{scra}$	Weak	$\mathbf{m_i}$	89%	90%
	Better	$\mathbf{m_i}^{sem}$	Weak	$\mathbf{m_i}$	84%	46%
	Better	$\mathbf{m_i}^{sem}$	Weak	$\mathbf{m_{i+1}}$	70%	71%
	Better	$\mathbf{m_i}^{sem}$	Better	$\mathbf{m_i}^{scra}$	44%	9%
	Weak	$\mathbf{m_{i+1}}$	Weak	$\mathbf{m_i}$	74%	29%

Table 6: Winrate scored by RM between the answers, comparing answers from Setting A to Setting B. We compare responses from Better in two settings (generation from scratch $\mathbf{m_i^{scra}}$, and generation with feedback from user ($\mathbf{m_i^{sem}}$). For Weak LLMs, where the original conversation is derived, we compare the initial model response $\mathbf{m_i}$ and the model response after user feedback $\mathbf{m_{i+1}}$. We empirically show: 1. Weak models could fail to address user feedback. 2. User-written instructions are imperfect. 3. Human feedback may not always help improve the model's response and the quality can vary across subsets and datasets.

E Comparison between MTBench and WildBench Prompts

For MTBench and WildBench, we compare the differences of prompt length, complexity and more in Table 7.To measure complexity score, we follow (Wang et al., 2024) to prompt GPT-4o-mini with questions and rubrics to get a score between 1 and 5, where high scores mean harder prompts.

Data	# prompts	Avg # tokens	complx.
MTBench	80	91.55	3.85
WildBench	1024	499.25	4.31

Table 7: Wildbench contains longer and more complex questions compared to MTBench.

F Comparison between MTBench Judge and WildBench Judge

We compare the scores from Judges released in MTBench and WildBench in Table 8.

Train Data	Split	Method	MT-JUDGE SCORE ↑	WILD-JUDGE SCORE ↑
WildChat	$\mathbb{D}^{\text{rand}}$	SFT on $\mathbf{m}_{\mathbf{i}}^{\mathbf{scra}}$	30.51 ± 2.43	4.62 ± 0.95
	$\mathbb{D}^{\text{neg}}$	SFT on $\mathbf{m}_{\mathbf{i}}^{\mathbf{scra}}$	31.08 ± 2.37	4.80 ± 1.69
	$\mathbb{D}^{\text{neg}}$	SFT on $\mathbf{m}_{\mathbf{i}}^{\mathbf{sem}}$	27.08 ± 1.29	0.1 ± 1.17

Table 8: Comparison of Vicuna evaluation results by MT-Judge (LLM Judge from MT-Bench) and Wild-Judge (LLM Judge from WildBench).

G Statistical Significance Test

We perform t-tests over the SFT on $\mathbb{D}^{\text{rand}}$ and two other baselines on $\mathbb{D}^{\text{neg}}$. As shown in Table 9, our significance test results show that our main take-aways are statistically significant. Specifically:

1. On MTBench, all the comparisons are statistically significant, confirming that providing targeted fixes with feedback semantics can help models improve on MTBench.
2. WildBench results are more mixed, as we discuss in the main paper:
   (a) Finetuning on $\mathbb{D}^{neg}$ does not consistently outperform $\mathbb{D}^{rand}$.
   (b) Finetuning on $\mathbb{D}^{sem}$significantly underperforms $\mathbb{D}^{neg}$, which aligns with our hypothesis and reward model analysis that regenerating with targeted feedback semantics is not always more effective than regenerating from scratch.

Train Data	Split	Method	MT-BENCH P-VALUES		WILDBENCH P-VALUES	
			Vicuna-7b	Mistral-7B	Vicuna-7b	Mistral-7B
LMSYS	$\mathbb{D}^{\text{rand}}$	SFT on $\mathbf{m}_i^{scra}$	*(reference)*		*(reference)*	
	$\mathbb{D}^{\text{neg}}$	SFT on $\mathbf{m}_i^{scra}$	0.00543	0.000889	0.445797	0.488326
	$\mathbb{D}^{\text{neg}}$	SFT on $\mathbf{m}_i^{sem}$	$<.00001$	$<.00001$	0.012582	0.000834
WildChat	$\mathbb{D}^{\text{rand}}$	SFT on $\mathbf{m}_i^{scra}$	*(reference)*		*(reference)*	
	$\mathbb{D}^{\text{neg}}$	SFT on $\mathbf{m}_i^{scra}$	0.000279	0.000027	0.184309	0.000264
	$\mathbb{D}^{\text{neg}}$	SFT on $\mathbf{m}_i^{sem}$	$<.00001$	$<.00001$	0.000369	$<.00001$

Table 9: Statistical significance test results (p-values) from paired t-tests comparing each method against SFT on $\mathbf{m}_i^{scra}$ with $\mathbb{D}^{\text{rand}}$ split.

H Feedback Detection

H.1 Feedback Distribution

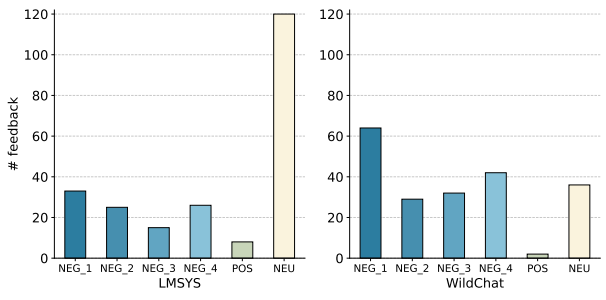


Figure 6: Distribution of dense human annotated labels.

We present the distribution of our annotated feedback categories in Fig 6.

H.2 Prompts

Context
You will be given a multi-turn conversation between a User and an Assistant. You should act as a human annotator to identify User feedback for the Assistant. Please read the conversation and complete the task below.
Task
Your task is to identify all feedback instances for Assistant in the User responses that satisfy the following feedback patterns:
Repeat or Rephrase (NEG_1)
Does the user repeat or rephrase their concern?
Examples for "yes":
- By house, I mean apartments, not condo
- Actually, I wanted

Examples for "no":
- Thank you

...
Format
You should output annotations per User turn except for the first query. You should both output the content of the User turn where feedback exists as well as the feedback pattern category using a json format:

```
{
"User Response Pattern": [Insert User Response Pattern],
"User Response Text": [Insert User Response Text]
}
If there's no feedback, please output: {
"User Response Pattern": "NEU",
"User Response Text": [Insert User Response Text]
}
```

Here are four examples of an input and your expected output.
...
Now you try:
Input:

Table 10: Prompt for feedback detection.

H.3 Feedback Detection Performance

We present the detailed scores of feedback detection performance across Sparse and Dense Eval sets in Table 11,12,13,14,15, 16.

Metric	Theirs (%)	Ours (%)
False positives	7.76	0.00
False negatives	50.86	18.86
True positives	41.38	42.29
True negatives	0.00	38.86
Accuracy	41.38	81.14
Recall	44.86	69.16
Precision	84.21	100.00

Table 11: Binary Detection performance on Sparse eval set.

Metric	Theirs	Ours
# predicted feedback / conversation	1.1	2.11
False positives (%)	7.17	15.32
False negatives (%)	61.36	43.07
True positives (%)	22.71	24.09
True negatives (%)	8.77	17.52
Accuracy (%)	31.47	41.61
Recall (%)	27.01	35.87
Precision (%)	76.00	61.11

Table 12: Binary Detection performance on Dense eval set.

Class	P (%)	R (%)	F1 (%)
POS	66.67	50.00	57.14
NEG	80.43	37.37	51.03
NEU	24.71	70.00	36.52
Accuracy	45.26	45.26	45.26
Macro avg	57.27	52.46	48.23
Weighted avg	67.43	45.26	48.21

Table 13: Three-way classification (theirs) on Sparse eval. "P", "R", and "F1" stand for precision, recall and F1-score respectively.

Class	Precision (%)	Recall (%)	F1-Score (%)
NEG	68.82	55.65	61.54
NEU	52.73	66.67	58.88
POS	62.50	55.56	58.82
Accuracy	60.19	60.19	60.19
Macro avg	61.35	59.29	59.75
Weighted avg	61.91	60.19	60.33

Table 14: Three-way classification (ours) on Sparse eval.

Class	Precision (%)	Recall (%)	F1-Score (%)
NEG	18.70	70.49	29.55
NEU	69.64	17.11	27.46
POS	70.00	100.00	82.35
Accuracy	30.07	30.07	30.07
Macro avg	52.78	62.53	46.46
Weighted avg	59.15	30.07	29.19

Table 15: Three-way classification (theirs) on Dense eval.

Class	Precision (%)	Recall (%)	F1-Score (%)
NEG	29.92	58.22	39.53
NEU	79.55	54.65	64.79
POS	25.81	32.00	28.57
Accuracy	55.35	55.35	55.35
Macro avg	45.09	48.29	44.30
Weighted avg	66.97	55.35	58.32

Table 16: Three-way classification (ours) on Dense eval.